Zen-Voyager:

Autonomous Exploration and Navigation in Open Worlds

Hanzo AI Research[1] **Zoo Labs Foundation**[2]

[1]Hanzo AI Inc. (Techstars '17) [2]Zoo Labs Foundation (501(c)(3))

research@hanzo.ai foundation@zoo.ngo

https://hanzo.ai/research/zen-voyager

February 2026

Abstract

We present **Zen-Voyager**, a foundation model for autonomous exploration and navigation in open-world environments. Zen-Voyager integrates curiosity-driven exploration with hierarchical goal planning and real-time semantic mapping to enable embodied agents that can navigate, understand, and reason about previously unseen environments without task-specific fine-tuning. Our architecture combines a vision-language backbone derived from the Zen model family with a novel Curiosity-Guided Exploration Module (CGEM) that computes intrinsic motivation signals from prediction errors in a learned world model. We introduce Semantic Occupancy Graphs (SOGs), a unified spatial representation that jointly encodes geometric occupancy, semantic labels, and object affordances, enabling both metric and topological planning. Zen-Voyager achieves state-of-the-art results on Habitat ObjectNav (87.3% success rate), AI2-THOR RoomNav (91.2% SPL), and the Open-World Navigation Benchmark (OWNB), outperforming prior methods by 12–18% in zero-shot transfer. We further demonstrate successful sim-to-real transfer on a Boston Dynamics Spot robot navigating office and warehouse environments with no additional training, achieving 82.7% success on 150 real-world navigation episodes. All models, code, and benchmark extensions are released under Apache 2.0.

Keywords: Embodied AI, Curiosity-Driven Exploration, Semantic Mapping, Hierarchical Planning, Sim-to-Real Transfer, Open-World Navigation

1 Introduction

Autonomous navigation in open-world environments represents a fundamental challenge in embodied artificial intelligence. Unlike constrained navigation tasks where agents

operate within known maps or follow pre-defined waypoints, open-world navigation requires agents to explore unfamiliar territories, build internal representations on the fly, and reason about spatial relationships, object semantics, and navigational affordances simultaneously.

Human navigators accomplish these tasks effortlessly. When entering an unfamiliar building, a person rapidly constructs a mental model of the space, identifies landmarks, infers room connectivity from architectural cues, and plans routes to desired destinations—all while maintaining awareness of previously visited areas and unexplored frontiers. Replicating this capability in artificial agents has proven remarkably difficult, as it requires the seamless integration of perception, memory, planning, and motor control.

Prior approaches to embodied navigation have made significant progress along individual dimensions. Learning-based methods (Anderson et al., 2018b; Chaplot et al., 2020) train end-to-end policies that map observations directly to actions, achieving strong performance on specific benchmarks but often failing to generalize across environments. Classical mapping and planning methods (Thrun et al., 2005; Kavraki et al., 1996) provide robust navigation in known environments but struggle with the perceptual complexity of real-world scenes. More recently, large vision-language models have been applied to navigation (Shah et al., 2023; Huang et al., 2023), demonstrating impressive semantic reasoning but lacking the spatial precision required for reliable collision-free motion.

Zen-Voyager addresses these limitations through three core contributions:

1. **Curiosity-Guided Exploration Module (CGEM):** A learned intrinsic motivation system that drives the agent toward informative observations by computing prediction errors in a world model. Unlike prior curiosity methods (Pathak et al., 2017; Burda et al., 2019), CGEM operates on semantic features rather than pixel-level predictions, making it robust to visual stochasticity while remaining sensitive to meaningful environmental structure.

2. **Semantic Occupancy Graphs (SOGs):** A unified spatial representation that combines metric occupancy grids with semantic labels and topological connectivity. SOGs are incrementally constructed from egocentric observations and support both local obstacle avoidance and global route planning, bridging the gap between geometric and semantic spatial reasoning.

3. **Hierarchical Goal Planning (HGP):** A three-level planning hierarchy that decomposes navigation objectives into subgoal sequences. The high-level planner uses the vision-language backbone to interpret natural language goals and identify target regions in the SOG. The mid-level planner generates frontier-based waypoints. The low-level controller executes point-to-point navigation with dynamic obstacle avoidance.

We evaluate Zen-Voyager on four established benchmarks—Habitat ObjectNav (Batra et al., 2020), AI2-THOR RoomNav (Kolve et al., 2017), the Matterport3D point-goal task (Chang et al., 2017), and our newly proposed Open-World Navigation Benchmark

(OWNB)—as well as 150 real-world navigation episodes on a quadruped robot. Across all settings, Zen-Voyager establishes new state-of-the-art results, demonstrating that a single model can achieve robust navigation across diverse environments without task-specific adaptation.

2 Background and Related Work

2.1 Embodied Navigation

The embodied navigation problem requires an agent situated in a 3D environment to reach target locations specified by coordinates, object categories, natural language descriptions, or images. Formally, at each time step t, the agent receives an observation o_t (typically RGB-D images) and must select an action a_t from a discrete set $\mathcal{A} = \{\texttt{forward}, \texttt{left}, \texttt{right}, \texttt{stop}\}$ to navigate toward a goal g.

Early deep learning approaches trained recurrent neural networks end-to-end via reinforcement learning (Zhu et al., 2017; Mirowski et al., 2017). While these methods demonstrated that neural networks can learn navigation policies, they suffered from poor sample efficiency, limited generalization, and catastrophic forgetting when transferred to new environments.

2.2 Modular Navigation Systems

Modular approaches decompose navigation into perception, mapping, and planning stages. SemExp (Chaplot et al., 2020) introduced a semantic exploration framework that builds top-down semantic maps and uses frontier-based exploration. PONI (Ramakrishnan et al., 2022) predicts potential functions over semantic maps to guide exploration. These methods achieve stronger generalization than end-to-end approaches but rely on hand-designed heuristics for exploration and goal selection.

2.3 Curiosity-Driven Exploration

Intrinsic curiosity (Pathak et al., 2017) augments extrinsic rewards with prediction-error-based intrinsic motivation. Random Network Distillation (RND) (Burda et al., 2019) uses the prediction error of a randomly initialized network as a novelty signal. Count-based methods (Bellemare et al., 2016) maintain visitation counts in learned feature spaces. While effective in game environments, these methods often fail in visually complex 3D environments due to the "noisy TV" problem—high prediction error from stochastic visual elements that carry no navigational information.

2.4 Vision-Language Navigation

Vision-Language Navigation (VLN) (Anderson et al., 2018a) introduced the task of following natural language instructions in photorealistic environments. Subsequent work explored pre-training (Hao et al., 2020), graph-based planning (Chen et al., 2022), and foundation model integration (Shah et al., 2023). LM-Nav (Shah et al., 2023) combines

large language models with vision-language models for image-goal navigation, achieving impressive results but lacking spatial precision for complex environments.

2.5 Sim-to-Real Transfer

Transferring navigation policies from simulation to the real world remains challenging due to the domain gap in visual appearance, physics, and sensor noise (Tobin et al., 2017; Sadeghi and Levine, 2017). Domain randomization (Tobin et al., 2017), system identification (Tan et al., 2018), and adversarial training (Pinto et al., 2017) have been proposed to close this gap. Recent work on neural radiance fields (Adamkiewicz et al., 2022) and Gaussian splatting (Kerbl et al., 2023) provides more photorealistic simulation environments, narrowing but not eliminating the transfer gap.

3 Architecture

Zen-Voyager consists of four tightly integrated modules: (1) a Vision-Language Perception Backbone, (2) the Curiosity-Guided Exploration Module (CGEM), (3) the Semantic Occupancy Graph (SOG) builder, and (4) the Hierarchical Goal Planner (HGP). Figure **??** provides an architectural overview.

3.1 Vision-Language Perception Backbone

The perception backbone processes egocentric RGB-D observations into dense semantic feature maps. We employ a dual-encoder architecture consisting of a visual encoder f_v and a language encoder f_l, both initialized from the Zen-72B vision-language model and fine-tuned for embodied perception tasks.

Visual Encoder. Given an RGB image $I_t \in \mathbb{R}^{H \times W \times 3}$ and depth map $D_t \in \mathbb{R}^{H \times W}$ at time t, the visual encoder produces a spatial feature map:

$$F_t = f_v(I_t, D_t) \in \mathbb{R}^{h \times w \times d} \tag{1}$$

where $h = H/16$, $w = W/16$, and $d = 1024$. We use a Vision Transformer (ViT-L/16) architecture with depth tokens concatenated to RGB patch embeddings, following Ranftl et al. (2021). The depth information is encoded as learnable Fourier positional features:

$$\phi(D_t) = [\sin(2\pi\sigma_k D_t), \cos(2\pi\sigma_k D_t)]_{k=1}^{K} \tag{2}$$

where σ_k are log-linearly spaced frequencies and $K = 64$.

Language Encoder. For goal specifications provided in natural language (e.g., "navigate to the kitchen"), the language encoder produces a goal embedding:

$$g = f_l(\text{tokenize}(s)) \in \mathbb{R}^d \tag{3}$$

where s is the goal string. This embedding is used by the hierarchical planner to ground navigation targets in the semantic map.

Cross-Modal Fusion. Visual and language features are fused through cross-attention layers:

$$\tilde{F}_t = \text{CrossAttn}(F_t, g) = \text{softmax}\left(\frac{(F_t W_Q)(g W_K)^\top}{\sqrt{d}}\right)(g W_V) \tag{4}$$

producing goal-conditioned visual features $\tilde{F}_t$ that highlight regions relevant to the current navigation objective.

3.2 Curiosity-Guided Exploration Module (CGEM)

The CGEM generates intrinsic motivation signals that drive the agent to explore informative regions of the environment. Unlike pixel-level curiosity (Pathak et al., 2017), CGEM operates on semantic features, making it robust to visual noise while remaining sensitive to novel objects, room transitions, and navigational affordances.

World Model. The CGEM maintains a predictive world model $\mathcal{W}_\theta$ that forecasts the semantic features the agent will observe after taking an action:

$$\hat{F}_{t+1} = \mathcal{W}_\theta(F_t, a_t, M_t) \tag{5}$$

where F_t is the current feature map, a_t is the selected action, and M_t is the current state of the semantic map. The world model is implemented as a transformer with 12 layers and 768-dimensional hidden states, trained online via self-supervised prediction.

Intrinsic Reward. The intrinsic reward at time t is computed as the prediction error in semantic feature space:

$$r_t^{\text{int}} = \left\| \hat{F}_{t+1} - \text{sg}(F_{t+1}) \right\|_2^2 \cdot \alpha(t) \tag{6}$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator and $\alpha(t)$ is a decay schedule that gradually reduces exploration as the environment becomes familiar:

$$\alpha(t) = \alpha_0 \cdot \exp\left(-\frac{t}{\tau}\right) + \alpha_{\min} \tag{7}$$

with $\alpha_0 = 1.0$, $\tau = 500$, and $\alpha_{\min} = 0.1$.

Semantic Novelty Filter. To address the noisy TV problem, we apply a semantic novelty filter that down-weights prediction errors from regions classified as dynamic or stochastic (e.g., flickering screens, moving shadows):

$$\tilde{r}_t^{\text{int}} = r_t^{\text{int}} \cdot (1 - p_{\text{dynamic}}(F_{t+1})) \tag{8}$$

Algorithm 1 Curiosity-Guided Exploration Module

Require: Perception backbone f_v, world model $\mathcal{W}_\theta$, semantic filter p_{dynamic}

Initialize replay buffer $\mathcal{B} \leftarrow \emptyset$
for $t = 1, 2, \ldots, T$ **do**
  Observe RGB-D pair (I_t, D_t)
  Compute features $F_t \leftarrow f_v(I_t, D_t)$
  Predict next features $\hat{F}_{t+1} \leftarrow \mathcal{W}_\theta(F_t, a_{t-1}, M_t)$
  Compute prediction error $e_t \leftarrow \|\hat{F}_t - F_t\|_2^2$
  Apply semantic filter $\tilde{e}_t \leftarrow e_t \cdot (1 - p_{\text{dynamic}}(F_t))$
  Compute intrinsic reward $r_t^{\text{int}} \leftarrow \tilde{e}_t \cdot \alpha(t)$
  Store (F_t, a_{t-1}, F_{t+1}) in $\mathcal{B}$
  if $|\mathcal{B}| \geq B_{\min}$ and $t \mod U = 0$ **then**
    Sample batch from $\mathcal{B}$ and update $\mathcal{W}_\theta$
  end if
end for

where p_{dynamic} is a learned binary classifier trained to distinguish static structural features from dynamic visual elements.

3.3 Semantic Occupancy Graphs

The Semantic Occupancy Graph (SOG) is a unified spatial representation that jointly encodes geometric occupancy, semantic labels, and topological connectivity. Unlike conventional occupancy grids or topological maps, SOGs bridge metric and semantic spatial reasoning within a single data structure.

Representation. An SOG is defined as $\mathcal{G} = (V, E, \Phi)$ where:

- $V = \{v_i\}$ is a set of nodes, each representing a voxelized region of space at resolution $\delta = 0.05\text{m}$
- $E \subseteq V \times V$ is a set of edges encoding navigability between adjacent nodes
- $\Phi : V \to \mathbb{R}^{d_s}$ assigns a semantic embedding vector to each node

Each node v_i carries the following attributes:

$$v_i = (x_i, y_i, z_i, o_i, s_i, c_i, \phi_i) \tag{9}$$

where (x_i, y_i, z_i) are metric coordinates, $o_i \in [0, 1]$ is the occupancy probability, $s_i \in \mathcal{S}$ is a semantic label, $c_i \in \mathbb{N}$ is a visitation count, and $\phi_i \in \mathbb{R}^{d_s}$ is a dense semantic embedding.

Incremental Construction. At each time step, the SOG is updated from the current RGB-D observation using the following pipeline:

1. **Point Cloud Generation:** Back-project the depth map D_t into a 3D point cloud P_t using the known camera intrinsics K.

2. **Pose Estimation:** Estimate the agent's pose $T_t \in SE(3)$ via visual-inertial odometry, with loop closure detection for drift correction.

3. **Semantic Projection:** Project the feature map F_t onto the point cloud, assigning semantic embeddings to each 3D point.

4. **Voxel Integration:** Integrate the labeled point cloud into the SOG using a running average for occupancy and semantic embeddings:

$$o_i^{(t)} = \frac{c_i^{(t-1)} \cdot o_i^{(t-1)} + o_i^{\text{obs}}}{c_i^{(t-1)} + 1}, \quad \phi_i^{(t)} = \frac{c_i^{(t-1)} \cdot \phi_i^{(t-1)} + \phi_i^{\text{obs}}}{c_i^{(t-1)} + 1} \tag{10}$$

5. **Edge Update:** Add navigability edges between adjacent free-space nodes using a traversability classifier.

Frontier Detection. Frontiers—boundaries between explored and unexplored space—are identified as nodes with $c_i > 0$ that are adjacent to nodes with $c_j = 0$:

$$\mathcal{F}_t = \{v_i \in V : c_i > 0 \wedge \exists v_j \in \text{adj}(v_i) : c_j = 0\} \tag{11}$$

These frontiers serve as candidate waypoints for exploration, ranked by the CGEM's predicted information gain.

3.4 Hierarchical Goal Planner

The Hierarchical Goal Planner operates at three temporal scales to decompose abstract navigation goals into executable motor commands.

High-Level Planner (Semantic). Given a natural language goal s or object category c, the high-level planner identifies target regions in the SOG by computing semantic similarity between the goal embedding g and node embeddings:

$$\text{score}(v_i) = \frac{g^\top \phi_i}{\|g\| \cdot \|\phi_i\|} + \beta \cdot \text{reachability}(v_i) \tag{12}$$

where $\text{reachability}(v_i)$ is the estimated traversal cost from the agent's current position to v_i, computed via Dijkstra's algorithm on the SOG, and $\beta = 0.3$ balances semantic relevance with spatial proximity.

When the target object has not yet been observed, the planner predicts likely locations using a learned spatial prior:

$$p(c|v_i) = \text{MLP}(\phi_i, \text{room_type}(v_i), \text{rel_pos}(v_i)) \tag{13}$$

This prior captures commonsense knowledge such as "refrigerators are typically found in kitchens" and "beds are in bedrooms."

Mid-Level Planner (Frontier). The mid-level planner selects the next frontier waypoint to explore, balancing the CGEM's predicted information gain with progress toward the high-level goal:

$$w^* = \arg\max_{w \in \mathcal{F}_t} \left[\lambda_1 \cdot r^{\text{int}}(w) + \lambda_2 \cdot \text{goal_progress}(w) + \lambda_3 \cdot \text{area_coverage}(w) \right] \tag{14}$$

where $\lambda_1 = 0.4$, $\lambda_2 = 0.5$, $\lambda_3 = 0.1$ are mixing weights, $r^{\text{int}}(w)$ is the predicted intrinsic reward at frontier w, and goal_progress(w) measures expected reduction in distance to the target.

Low-Level Controller (Motor). The low-level controller executes point-to-point navigation between waypoints using a learned local policy:

$$a_t = \pi_\phi(F_t, D_t, w_{\text{local}}, v_t) \tag{15}$$

where w_{local} is the current local waypoint and v_t is the agent's velocity. This policy is trained with PPO (Schulman et al., 2017) in simulation to handle dynamic obstacle avoidance, narrow passages, and uneven terrain.

4 Training

4.1 Pre-training

Zen-Voyager's training follows a three-stage curriculum: perception pre-training, exploration pre-training, and navigation fine-tuning.

Stage 1: Perception Pre-training. The vision-language backbone is initialized from the Zen-72B checkpoint and fine-tuned on 2.4 million indoor scene images with dense semantic annotations from ScanNet (Dai et al., 2017), Matterport3D (Chang et al., 2017), and HM3D (Ramakrishnan et al., 2021). We train for 50 epochs with AdamW (lr $= 2 \times 10^{-5}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $= 0.01$) and a cosine learning rate schedule with 2000 warm-up steps.

Stage 2: Exploration Pre-training. The CGEM and world model are trained on 10,000 procedurally generated environments using ProcTHOR (Deitke et al., 2022). The agent explores each environment for 500 steps, collecting trajectories that are used to train the world model via self-supervised next-step prediction:

$$\mathcal{L}_{\text{world}} = \mathbb{E}_{(F_t, a_t, F_{t+1}) \sim \mathcal{B}} \left[\| \mathcal{W}_\theta(F_t, a_t, M_t) - F_{t+1} \|_2^2 \right] \tag{16}$$

Simultaneously, the exploration policy is trained with PPO using a combination of intrinsic and coverage rewards:

$$r_t = \tilde{r}_t^{\text{int}} + \gamma_{\text{cov}} \cdot \Delta\text{coverage}_t \tag{17}$$

where $\Delta\text{coverage}_t$ is the increase in explored area at time t and $\gamma_{\text{cov}} = 0.5$.

Stage 3: Navigation Fine-tuning. The full system is fine-tuned on target navigation tasks using a combination of imitation learning and reinforcement learning. We use DAgger (Ross et al., 2011) with an oracle planner that has access to the ground-truth map, gradually transitioning to the agent's own predictions:

$$\mathcal{L}_{\text{nav}} = (1 - \epsilon_t) \cdot \mathcal{L}_{\text{IL}} + \epsilon_t \cdot \mathcal{L}_{\text{RL}} \tag{18}$$

where ϵ_t increases linearly from 0 to 1 over 10,000 episodes. The RL loss uses PPO with a reward:

$$r_t^{\text{nav}} = \begin{cases} 10.0 & \text{if goal reached} \\ -0.01 & \text{per step (time penalty)} \\ \Delta d_t & \text{progress reward} \end{cases} \tag{19}$$

where $\Delta d_t = d_{t-1}^{\text{goal}} - d_t^{\text{goal}}$ is the reduction in geodesic distance to the goal.

4.2 Training Infrastructure

Training is distributed across 64 NVIDIA A100 GPUs organized in 8 nodes of 8 GPUs each. We use a hybrid parallelism strategy:

- **Data parallelism** across nodes for the perception backbone
- **Environment parallelism** with 32 parallel Habitat/AI2-THOR instances per GPU for trajectory collection
- **Asynchronous updates** between simulation rollouts and gradient computation to maximize GPU utilization

The full training pipeline requires approximately 72 hours (Stage 1: 8h, Stage 2: 40h, Stage 3: 24h). We use mixed-precision training (BF16) with gradient checkpointing to fit the model and replay buffer in GPU memory.

4.3 Data Augmentation and Domain Randomization

To improve sim-to-real transfer, we apply extensive domain randomization during training:

- **Visual randomization:** Random textures, lighting conditions (intensity, color temperature, shadow direction), camera noise (Gaussian, salt-and-pepper, motion blur), and post-processing effects (brightness, contrast, saturation jitter).
- **Physical randomization:** Agent dimensions ($\pm 10\%$), step size ($\pm 15\%$), turn angle ($\pm 5°$), and sensor mounting height (± 5cm).
- **Environmental randomization:** Furniture placement, door states (open/closed/-partially open), clutter density, and floor material (affecting traversability).

- **Depth noise model:** We simulate realistic depth sensor noise based on the Intel RealSense D435 noise profile, including quantization, edge artifacts, and missing measurements.

5 Sim-to-Real Transfer

Bridging the simulation-to-reality gap is critical for deploying Zen-Voyager on physical robots. We address this challenge through a combination of robust representations, domain randomization (Section 4.3), and an adaptation module.

5.1 Robust Feature Representations

The semantic features learned by the perception backbone are inherently more robust to domain shift than pixel-level representations. We quantify this by measuring the Maximum Mean Discrepancy (MMD) between features extracted from simulated and real environments:

$$\mathrm{MMD}^2(\mathcal{F}_{\text{sim}}, \mathcal{F}_{\text{real}}) = \left\| \frac{1}{n} \sum_{i=1}^{n} \phi(x_i^{\text{sim}}) - \frac{1}{m} \sum_{j=1}^{m} \phi(x_j^{\text{real}}) \right\|_{\mathcal{H}}^2 \tag{20}$$

Our semantic features achieve an MMD of 0.023, compared to 0.187 for pixel-level features and 0.089 for pre-trained ImageNet features, confirming that semantic abstraction reduces the domain gap.

5.2 Real-Time Adaptation Module

For deployment, we introduce a lightweight adaptation module that fine-tunes the first two layers of the visual encoder on a small set of unlabeled real-world images (approximately 100 images, collected in 5 minutes of manual exploration):

$$\mathcal{L}_{\text{adapt}} = \mathcal{L}_{\text{contrast}}(f_v^{\text{sim}}, f_v^{\text{real}}) + \lambda_{\text{consist}} \cdot \mathcal{L}_{\text{consistency}} \tag{21}$$

where $\mathcal{L}_{\text{contrast}}$ is a contrastive loss aligning simulated and real feature distributions, and $\mathcal{L}_{\text{consistency}}$ ensures temporal consistency between consecutive real frames. This adaptation takes less than 10 minutes on a single GPU.

5.3 Hardware Platform

We deploy Zen-Voyager on a Boston Dynamics Spot quadruped robot equipped with:

- Intel RealSense D435 RGB-D camera (640×480, 30 fps)
- NVIDIA Jetson AGX Orin for on-board inference
- Custom ROS 2 integration for motor commands and sensor data

The full inference pipeline runs at 12 Hz on the Jetson AGX Orin, with the perception backbone consuming 60ms, SOG update 8ms, and planner 15ms per frame. The remaining time is used for sensor synchronization and communication overhead.

6 Evaluation

We evaluate Zen-Voyager across four simulation benchmarks and one real-world benchmark, comparing against state-of-the-art baselines in each setting.

6.1 Simulation Benchmarks

Habitat ObjectNav. The agent must navigate to an instance of a specified object category in photorealistic Matterport3D environments. We evaluate on the standard val split (2195 episodes, 6 object categories).

AI2-THOR RoomNav. The agent navigates to a specified room type (kitchen, bedroom, bathroom, living room) in procedurally generated apartments. We use the ProcTHOR-10K evaluation set.

MP3D Point-Goal. A classical point-goal navigation task in Matterport3D environments where the agent receives the relative displacement to the goal. We evaluate on the standard val split.

Open-World Navigation Benchmark (OWNB). Our newly proposed benchmark evaluates navigation in environments with novel object categories, dynamic obstacles, and natural language goals of varying specificity. OWNB contains 5000 episodes across 200 environments with 4 difficulty levels.

6.2 Metrics

- **Success Rate (SR):** Fraction of episodes where the agent reaches within 1.0m of the goal and calls `stop`.
- **Success weighted by Path Length (SPL)** (Anderson et al., 2018b): $\frac{1}{N}\sum_{i=1}^{N} S_i \cdot \frac{l_i}{\max(p_i, l_i)}$, where S_i is success, l_i is shortest path length, and p_i is actual path length.
- **SoftSPL:** A continuous relaxation of SPL using progress toward the goal rather than binary success.
- **Coverage (%):** Fraction of navigable area explored by the agent.
- **Exploration Efficiency (EE):** Coverage divided by number of steps, measuring how quickly the agent maps the environment.

Table 1: Results on Habitat ObjectNav (Matterport3D val split). Best results in **bold**, second best underlined.

Method	SR (%)	SPL (%)	SoftSPL (%)
Frontier-DD	42.1	23.8	31.4
SemExp	65.3	38.2	45.7
PONI	69.1	41.6	49.3
PIRLNav	72.8	44.3	52.1
LM-Nav	68.5	39.7	47.8
HomeRobot	71.2	42.8	50.6
Zen-Voyager (Ours)	**87.3**	**58.1**	**65.4**
w/o CGEM	79.6	50.3	57.8
w/o SOG (grid map)	81.2	52.7	59.1
w/o HGP (flat policy)	76.4	47.9	55.3

Table 2: Results on AI2-THOR RoomNav (ProcTHOR-10K evaluation set).

Method	SR (%)	SPL (%)	EE (%/step)
Frontier-DD	51.3	38.2	0.12
SemExp	72.6	55.4	0.18
PONI	76.8	58.1	0.21
PIRLNav	79.2	61.3	0.23
LM-Nav	75.4	54.9	0.19
Zen-Voyager (Ours)	**91.2**	**73.8**	**0.31**

6.3 Baselines

We compare against the following methods:

- **SemExp** (Chaplot et al., 2020): Semantic exploration with learned goal-oriented policies.
- **PONI** (Ramakrishnan et al., 2022): Potential function-based navigation with semantic priors.
- **PIRLNav** (Ramrakhya et al., 2023): Pre-training with internet-scale data for embodied navigation.
- **LM-Nav** (Shah et al., 2023): Large language model-guided navigation.
- **HomeRobot** (Yenamandra et al., 2023): Modular open-vocabulary mobile manipulation.
- **Frontier-DD** (Yamauchi, 1997): Classical frontier-based exploration with distance decay.

6.4 Simulation Results

Table 1 presents results on Habitat ObjectNav, where Zen-Voyager achieves 87.3% success rate and 58.1% SPL, outperforming the previous best (PIRLNav) by 14.5% and 13.8%

Table 3: Results on the Open-World Navigation Benchmark (OWNB) by difficulty level.

Method	Easy	Medium	Hard	Expert
SemExp	71.2	52.4	31.8	18.3
PIRLNav	76.8	58.1	37.2	22.6
LM-Nav	73.5	55.7	40.1	28.4
Zen-Voyager	**94.1**	**82.3**	**63.7**	**47.2**

Table 4: Real-world navigation results on Boston Dynamics Spot (150 episodes).

Environment	Episodes	SR (%)	SPL (%)
Office	60	85.0	62.3
Warehouse	50	80.0	58.1
Apartment	40	82.5	61.7
Overall	150	**82.7**	**60.8**

respectively. Ablation results confirm that each component contributes meaningfully: removing CGEM reduces SR by 7.7%, removing SOGs by 6.1%, and removing HGP by 10.9%.

Table 2 shows AI2-THOR RoomNav results, where Zen-Voyager achieves 91.2% SR and 73.8% SPL, with particularly strong exploration efficiency (0.31%/step vs. 0.23%/step for PIRLNav).

Table 3 demonstrates Zen-Voyager's robustness on the challenging OWNB benchmark. Performance advantages increase with task difficulty: on "Expert" episodes (requiring multi-room navigation with ambiguous goals and dynamic obstacles), Zen-Voyager outperforms the next best method by 18.8%.

6.5 Real-World Evaluation

We evaluate sim-to-real transfer through 150 navigation episodes in three real environments: a 500m^2 office building, a 300m^2 warehouse, and a 200m^2 residential apartment.

Table 4 shows that Zen-Voyager achieves 82.7% success rate across all real-world episodes, with a simulation-to-real performance gap of only 4.6% (compared to 87.3% on Habitat ObjectNav). The warehouse environment proved most challenging due to large open spaces with repetitive visual patterns and dynamic forklift traffic.

Failure Analysis. Among the 26 failed real-world episodes, we identified four primary failure modes:

1. **Glass surfaces** (8/26): Transparent glass walls and doors were not detected by the depth sensor, causing the agent to plan paths through obstacles.

2. **Dynamic obstacles** (7/26): Fast-moving people or vehicles that were not anticipated by the local planner.

Table 5: Comparison of curiosity formulations on Habitat ObjectNav.

Curiosity Method	SR (%)	Coverage (%)
No curiosity (random exploration)	58.3	42.1
ICM (Pathak et al., 2017)	72.1	61.3
RND (Burda et al., 2019)	74.8	64.7
CGEM (pixel-level)	78.2	71.3
CGEM (semantic, no filter)	83.6	78.9
CGEM (full)	**87.3**	**84.2**

Table 6: Impact of map representation on navigation performance.

Representation	SR	SPL	Memory	Update (ms)
2D Occupancy Grid	78.4	47.2	12 MB	3
3D Voxel Grid	80.1	49.8	256 MB	42
Topological Map	76.8	51.3	4 MB	1
Neural Implicit Map	82.3	52.1	64 MB	28
SOG (Ours)	**87.3**	**58.1**	48 MB	8

3. **Odometry drift** (6/26): Accumulated pose estimation errors in long corridors without distinctive visual features.

4. **Goal ambiguity** (5/26): Natural language goals with multiple valid interpretations (e.g., "go to the desk" in an open office with many desks).

7 Ablation Studies

We conduct comprehensive ablation studies to understand the contribution of each component and design choice.

7.1 Curiosity Variants

Table 5 demonstrates that semantic-level curiosity significantly outperforms pixel-level methods. The semantic novelty filter provides an additional 3.7% SR improvement by suppressing false novelty signals from dynamic visual elements.

7.2 Map Representation

We compare SOGs against alternative spatial representations:

SOGs achieve the best accuracy while maintaining reasonable memory usage and fast update times. The combined metric-semantic-topological nature of SOGs enables both fine-grained obstacle avoidance (from occupancy) and efficient global planning (from topology).

Table 7: Effect of backbone model size on Habitat ObjectNav performance.

Backbone Size	SR (%)	SPL (%)	Inference (ms)
Zen-600M	71.2	42.8	18
Zen-1.5B	76.8	48.3	32
Zen-7B	81.4	52.1	58
Zen-32B	85.7	56.4	145
Zen-72B	**87.3**	**58.1**	312

7.3 Scaling Analysis

We study how Zen-Voyager performance scales with the size of the vision-language backbone:

Performance improves log-linearly with model size, with diminishing returns above 32B parameters. For real-time deployment on edge devices, the Zen-7B variant provides an attractive accuracy-latency trade-off.

8 Open-World Navigation Benchmark

We introduce the Open-World Navigation Benchmark (OWNB) to evaluate navigation agents in realistic open-world conditions that go beyond existing benchmarks.

8.1 Benchmark Design

OWNB consists of 5000 episodes across 200 environments with four difficulty levels:

- **Easy (1250 episodes):** Single-room navigation with specific object goals and no dynamic obstacles.
- **Medium (1250 episodes):** Multi-room navigation with category-level goals (e.g., "find a chair") and static obstacles.
- **Hard (1250 episodes):** Multi-floor navigation with natural language goals (e.g., "go to where people eat") and dynamic obstacles (moving humans, pets).
- **Expert (1250 episodes):** Complex goals requiring inference (e.g., "find something to read in a quiet place"), adversarial dynamic obstacles, and partial environment changes between episodes.

8.2 Novel Evaluation Criteria

Beyond standard SR and SPL metrics, OWNB introduces:

- **Semantic Grounding Accuracy (SGA):** Whether the agent correctly identified the target object/location matching the natural language goal.
- **Safety Score:** Penalizes collisions with obstacles and near-misses with dynamic agents.

- **Replanning Efficiency:** Measures how quickly the agent recovers when blocked or redirected.

9 Discussion

9.1 Strengths

Zen-Voyager demonstrates several qualitative advantages over prior methods. The semantic-level curiosity mechanism avoids the noisy TV problem that plagues pixel-level curiosity, leading to more directed exploration. The SOG representation enables the agent to maintain a coherent spatial understanding that supports both local reactive navigation and global deliberative planning. The hierarchical planning architecture allows the system to handle goals at different levels of abstraction.

9.2 Limitations

Several limitations merit discussion:

- **Computational requirements:** The full Zen-72B backbone requires a high-end GPU for real-time inference, limiting deployment to robots with powerful onboard compute or reliable cloud connectivity.
- **Glass and mirrors:** Transparent and reflective surfaces remain challenging, as they produce incorrect depth readings that corrupt the SOG.
- **Long-horizon tasks:** For navigation tasks requiring more than 500 steps, odometry drift can accumulate to levels that degrade map quality, even with loop closure.
- **Social navigation:** The current system treats humans as dynamic obstacles rather than social agents, lacking the ability to navigate according to social norms (e.g., yielding in narrow corridors, maintaining personal space).

9.3 Future Directions

Several extensions are planned for future work:

1. **Multi-agent coordination:** Extending CGEM to support collaborative exploration by multiple robots, partitioning frontiers to minimize redundant coverage.
2. **Manipulation integration:** Combining navigation with mobile manipulation to enable tasks like "fetch a cup from the kitchen," requiring the agent to navigate, identify, grasp, and transport objects.
3. **Lifelong learning:** Enabling the agent to accumulate knowledge across multiple visits to the same or similar environments, building a persistent world model.
4. **Social awareness:** Training the agent to follow social navigation conventions using human trajectory datasets.

10 Ethical Considerations

Autonomous navigation systems raise important ethical considerations. Zen-Voyager is designed for benign applications such as assistive robotics, warehouse logistics, and search-and-rescue. We explicitly prohibit the use of our models and code for surveillance, autonomous weapons, or any application that infringes on individual privacy. The system does not store or transmit personally identifiable information from its observations. All real-world experiments were conducted with informed consent from building occupants, and face blurring was applied to all collected data before storage.

11 Conclusion

We presented Zen-Voyager, a foundation model for autonomous exploration and navigation in open-world environments. By integrating curiosity-driven exploration (CGEM), unified spatial representations (SOGs), and hierarchical goal planning (HGP) with a powerful vision-language backbone, Zen-Voyager achieves state-of-the-art performance across four simulation benchmarks and demonstrates successful zero-shot sim-to-real transfer on a physical robot. The system's 87.3% success rate on Habitat ObjectNav and 82.7% success rate in real-world navigation represent significant advances over prior methods.

We release all models, training code, the OWNB benchmark, and ROS 2 integration packages under Apache 2.0 at https://github.com/hanzoai/zen-voyager, and encourage the community to build upon this work toward truly autonomous navigation agents.

References

Anderson, P., Wu, Q., Teney, D., Bruce, J., Johnson, M., Sünderhauf, N., Reid, I., Gould, S., and van den Hengel, A. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3674–3683, 2018.

Anderson, P., Chang, A., Chaplot, D. S., Dosovitskiy, A., Gupta, S., Koltun, V., Kosecka, J., Malik, J., Mottaghi, R., Savva, M., and Zamir, A. R. On evaluation of embodied navigation agents. *arXiv preprint arXiv:1807.06757*, 2018.

Batra, D., Gokaslan, A., Kembhavi, A., Maksymets, O., Mottaghi, R., Jain, M., Wijmans, E., Galber, S., Straub, J., Savva, M., and Zhao, H. ObjectNav revisited: On evaluation of embodied agents navigating to objects. *arXiv preprint arXiv:2006.13171*, 2020.

Bellemare, M., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., and Munos, R. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, pp. 1471–1479, 2016.

Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.

Chang, A., Dai, A., Funkhouser, T., Halber, M., Niessner, M., Savva, M., Song, S., Zeng, A., and Zhang, Y. Matterport3D: Learning from RGB-D data in indoor environments. In *International Conference on 3D Vision*, pp. 667–676, 2017.

Chaplot, D. S., Gandhi, D., Gupta, S., Gupta, A., and Salakhutdinov, R. Learning to explore using active neural SLAM. In *International Conference on Learning Representations*, 2020.

Chaplot, D. S., Gandhi, D. P., Gupta, A., and Salakhutdinov, R. Object goal navigation using goal-oriented semantic exploration. In *Advances in Neural Information Processing Systems*, 2020.

Chen, S., Guhur, P.-L., Schmid, C., and Laptev, I. Think global, act local: Dual-scale graph transformer for vision-and-language navigation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 16537–16547, 2022.

Dai, A., Chang, A. X., Savva, M., Halber, M., Funkhouser, T., and Nießner, M. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5828–5839, 2017.

Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Ehsani, K., Salvador, J., Han, W., Kolve, E., Kembhavi, A., and Mottaghi, R. ProcTHOR: Large-scale embodied AI using procedural generation. In *Advances in Neural Information Processing Systems*, 2022.

Hao, W., Li, C., Li, X., Carin, L., and Gao, J. Towards learning a generic agent for vision-and-language navigation via pre-training. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 13137–13146, 2020.

Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International Conference on Machine Learning*, pp. 9118–9147, 2023.

Adamkiewicz, M., Chen, T., Caccavale, A., Gardner, R., Culbertson, P., Bohg, J., and Schwager, M. Vision-only robot navigation in a neural radiance world. *IEEE Robotics and Automation Letters*, 7(2):4606–4613, 2022.

Kavraki, L. E., Svestka, P., Latombe, J.-C., and Overmars, M. H. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.

Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023.

Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Gordon, D., Zhu, Y., Gupta, A., and Farhadi, A. AI2-THOR: An interactive 3D environment for visual AI. *arXiv preprint arXiv:1712.05474*, 2017.

Mirowski, P., Pascanu, R., Leber, F., Ghemawat, R., Dean, J., and Hadsell, R. Learning to navigate in complex environments. In *International Conference on Learning Representations*, 2017.

Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, pp. 2778–2787, 2017.

Pinto, L., Davidson, J., Sukthankar, R., and Gupta, A. Robust adversarial reinforcement learning. In *International Conference on Machine Learning*, pp. 2817–2826, 2017.

Ramakrishnan, S. K., Gokaslan, A., Wijmans, E., Maksymets, O., Clegg, A., Turner, J., Undersander, E., Galber, W., Chaplot, D. S., Swain, T., et al. Habitat-Matterport 3D Dataset (HM3D): 1000 large-scale 3D environments for embodied AI. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Ramakrishnan, S. K., Chaplot, D. S., Al-Halah, Z., Malik, J., and Grauman, K. PONI: Potential functions for ObjectGoal navigation with interaction-free learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 18890–18900, 2022.

Ramrakhya, R., Undersander, E., Batra, D., and Das, A. PIRLNav: Pretraining with imitation and RL finetuning for ObjectNav. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 17896–17906, 2023.

Ranftl, R., Bochkovskiy, A., and Koltun, V. Vision transformers for dense prediction. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 12179–12188, 2021.

Ross, S., Gordon, G., and Bagnell, D. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, pp. 627–635, 2011.

Sadeghi, F. and Levine, S. CAD2RL: Real single-image flight without a single real image. In *Robotics: Science and Systems*, 2017.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Shah, D., Osiński, B., Ichter, B., and Levine, S. LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action. In *Conference on Robot Learning*, pp. 492–504, 2023.

Tan, J., Zhang, T., Coumans, E., Iscen, A., Bai, Y., Hafner, D., Bohez, S., and Vanhoucke, V. Sim-to-real: Learning agile locomotion for quadruped robots. In *Robotics: Science and Systems*, 2018.

Thrun, S., Burgard, W., and Fox, D. *Probabilistic Robotics*. MIT Press, 2005.

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 23–30, 2017.

Yamauchi, B. A frontier-based approach for autonomous exploration. In *IEEE International Symposium on Computational Intelligence in Robotics and Automation*, pp. 146–151, 1997.

Yenamandra, S., Ramachandran, A., Yadav, K., Wang, A., Khanna, M., Gervet, T., Yang, T.-Y., Jain, V., Clegg, A., Turner, J., et al. HomeRobot: Open-vocabulary mobile manipulation. In *Conference on Robot Learning*, 2023.

Zhu, Y., Mottaghi, R., Kolve, E., Lim, J. J., Gupta, A., Fei-Fei, L., and Farhadi, A. Target-driven visual navigation in indoor scenes using deep reinforcement learning. In *IEEE International Conference on Robotics and Automation*, pp. 3357–3364, 2017.

A OWNB Benchmark Details

The Open-World Navigation Benchmark (OWNB) is constructed from 200 environments: 120 from Matterport3D, 50 from HM3D, and 30 custom-built environments using ProcTHOR. Environment statistics are provided in Table 8.

Table 8: OWNB environment statistics.

Statistic	**Mean**	**Min**	**Max**
Navigable area (m^2)	142.3	28.5	892.1
Number of rooms	8.7	2	42
Number of floors	1.8	1	4
Object categories	47.2	12	156
Dynamic agents	3.1	0	12

B Hyperparameter Details

C Computational Cost Analysis

With INT8 quantization of the visual encoder, total inference latency drops to 62ms (16 Hz), well within the real-time requirements for robotic navigation.

Table 9: Complete hyperparameter configuration for Zen-Voyager training.

Stage	Parameter	Value
Perception	Learning rate	2×10^{-5}
	Batch size	256
	Weight decay	0.01
	Warm-up steps	2000
	Total epochs	50
Exploration	PPO clip ϵ	0.2
	Discount γ	0.99
	GAE λ	0.95
	Entropy coefficient	0.01
	Parallel environments	256
	Steps per environment	500
Navigation	DAgger mixing start	0.0
	DAgger mixing end	1.0
	DAgger transition episodes	10,000
	Success reward	10.0
	Step penalty	-0.01

Table 10: Computational cost breakdown for Zen-Voyager inference on NVIDIA Jetson AGX Orin.

Module	Latency (ms)	Memory (MB)
Visual Encoder (Zen-7B)	42.3	4,200
Language Encoder	8.1	320
Cross-Modal Fusion	9.6	180
CGEM World Model	12.4	890
SOG Update	8.2	48
Hierarchical Planner	14.8	120
Total	**95.4**	**5,758**